An Area-Optimized 8-bit Deep Learning Accelerator for GAN Image Generation on GF180MCU

William Anthony, I Made Medika Surya W.M., John Ruben Saragi, Mahardhika Putra Adipratama,
Fairuz Aquilla Rahagi, Trio Adiono, and Infall Syafalni
Bandung Institute of Technology, Bandung, Indonesia
E-mail: {13223048, 13222021, 13222049, 13222081, 13222107}@mahasiswa.itb.ac.id,
tadiono@itb.ac.id, infall@ieee.org

***Abstract*—This paper presents an area-optimized 8-bit deep learning accelerator for a small GAN image generator, hardened through a fully open-source flow on the open GF180MCU 180 nm process. The network is a three-layer MNIST multi-layer-perceptron generator, quantized to signed 8-bit integers and mapped onto a structural 4×4 processing-element array of native accumulation depth $K = 256$. Starting from an eleven-macro predecessor, the result buffer is folded from three 256×8 SRAM macros used as byte planes into a single 64×8 macro addressed along the plane axis, cutting that buffer 77% and the macro count to nine on a 3×3 grid and a 1375×1325 μm die. The result is 24% less die area and 18% less power, with more setup margin, for 11.7% more compute cycles per image. Antenna closure needed a method the usual density sweep could not supply: the sweep plateaued at one residual net, but the violating net names identified a single SRAM macro whose column broadcast sat too far from the array, and swapping it with the small result buffer reached zero violations at two independent densities. The macro signs off with zero DRC, LVS, XOR, and antenna violations at 40 ns across nine STA corners, verified bit-exactly up to a full 784-pixel image through the routed netlist.**

***Index Terms*—Deep learning accelerator, GF180MCU, SRAM macro, area optimization, antenna closure, INT8 quantization, GAN, ASIC, LibreLane, open-source EDA.**

I. Introduction

Generative Adversarial Networks are a compact workload for testing neural inference on custom hardware. The generator used here is a small MNIST multi-layer perceptron turning a 64-dimensional latent vector into a 28×28 image; its arithmetic is dense matrix multiplication, so it maps onto a small array of multiply–accumulate units. An earlier version expressed this as behavioral loops and register memory arrays — convenient for checking the algorithm, but not implementable, since buffers inferred as standard-cell registers make the design too large to route. It was restructured into a structural accelerator with a processing-element (PE) array, SRAM-backed buffers, and a controller, and hardened with eleven 256×8 SRAM macros on a 1600×1500 μm die.

This paper questions that design's memory rather than accepting it. Three of its eleven macros held the result buffer as byte planes of one 24-bit word, storing 48 bytes in 768 — 6.25% occupancy, on a buffer the array geometry fixes at sixteen accumulators forever. Folding those planes onto the address axis of one 64×8 macro recovers 77% of that area and lets the die shrink. The contributions are: (1) a memory-folding transformation cutting die area 24% and power 18% while *improving* setup margin, at a measured cost in cycles; (2) an antenna-closure method for when a density sweep plateaus — read the violating net names to find a mis-placed macro instead of re-rolling placement — confirmed by two independent levers reaching zero; (3) a measured latency breakdown from per-stage cycle counts to end-to-end time over the host link; and (4) a measured justification of INT8 against INT4, INT16, and FP16.

II. Related Work

Most neural accelerators are arrays of multiply-and-accumulate units — the systolic TPU [1] at large scale, surveyed broadly in [2] — and running a floating-point network on an integer array normally uses the per-tensor quantization scale of Jacob et al. [6]. The fabricated reference points span three orders of magnitude: DianNao [3], Eyeriss [4] with its row-stationary dataflow, and, closer to our scale, Wang et al. [7], all closed-PDK designs on commercial nodes. The comparison that matters most is OpenSpike [5], a spiking accelerator in open SkyWater 130 nm with open EDA and OpenRAM memory: same open-flow premise, different workload and memory strategy (Table V). The workload is a pretrained MLP GAN [8], [9]; the RTL builds on the open DLA Engine project [10].

III. GAN Architecture

The generator (Fig. 1) projects a 64-dimensional latent vector through two 256-neuron ReLU hidden layers into a 784-neuron tanh output layer, reshaped as a 28×28 grayscale image. Its dense layers are L0 (256×64, ReLU), L2 (256×256, ReLU), and L4 (784×256, tanh): 282,624 MACs. At four output neurons per tile, L0 and L2 need 64 tiles each and L4 needs 196 — 324 tiles per image.

IV. Hardware Architecture

The DLA computes a tiled matrix multiplication $C = A\times B$ with array size $N = 4$ and native accumulation depth $K = 256$, so one transaction accumulates a full 256-term dot product — the inner dimension of the widest generator layers. K is temporal: it sets how many cycles the same 16

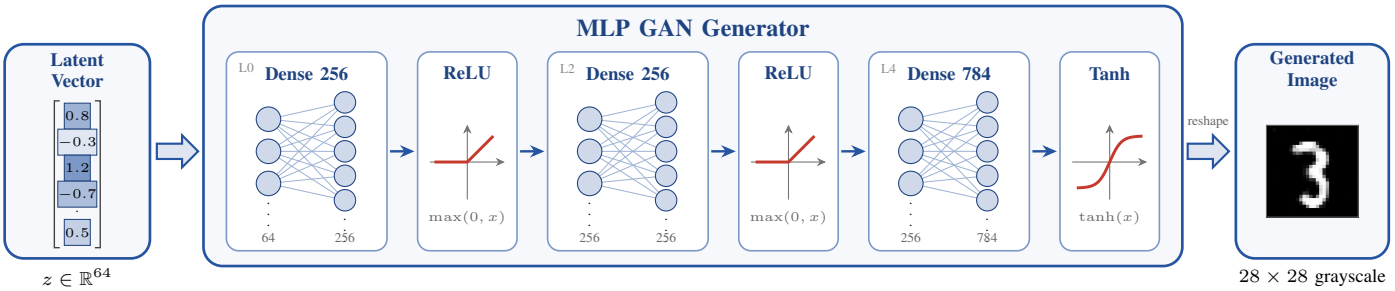

Fig. 1. MLP GAN generator: the 64-dimensional latent vector z is expanded through two 256-neuron ReLU hidden layers and a 784-neuron tanh output layer into a 28×28 grayscale image.

PEs accumulate, not the array size. An earlier RTL revision defaulted to $K = 4$ and relied on a harness override; since the physical flow synthesizes bare RTL defaults, the hardened default was widened to $K = 256$, a parameter-only change as the operand SRAMs were already 256 deep. Per transaction the controller clears the accumulators, enables the array for K cycles, and stores the result block to C through a writeback path that serializes the sixteen accumulators.

At the top level (Fig. 2) A provides one row vector to the array and B one column vector, the control unit generates the read index and the clear, enable, done, and busy signals, and the output bus carries the accumulated C values.

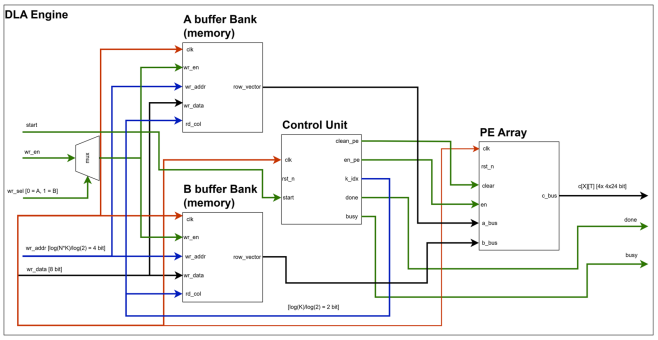

Fig. 2. DLA engine top level.

A. PE Array and Controller

Each PE (Fig. 3, left) takes two signed 8-bit operands, multiplies them, sign-extends the 16-bit product to 24 bits, and adds it to a local accumulator,

$$acc[n] = acc[n-1] + a[n]\,b[n], \tag{1}$$

so after K cycles it holds one tile output; 24 bits give 256 accumulated 8-bit products enough range. The array (Fig. 3, right) is 16 PEs in a 4×4 grid, each row fed one activation lane and each column one weight lane by broadcast, each PE accumulating locally — output-stationary, all 16 in parallel, one tile per accumulation loop. The controller (Fig. 4) has four states: IDLE waits for START, CLEAR zeroes the accumulators, COMPUTE enables the array and increments the A and B addresses until all K terms are consumed, DONE holds the completion flag until START falls. A writeback sequencer then serializes the accumulators into C and raises a writeback-done flag.

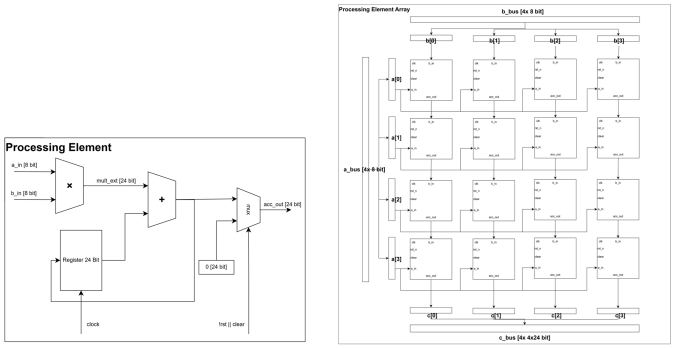

Fig. 3. Processing element datapath (left) and the 4×4 array (right).

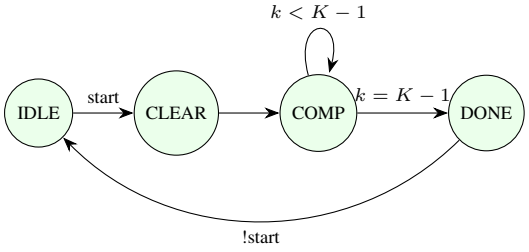

Fig. 4. Controller state machine.

B. SRAM Buffers

The operand buffers are single-port SRAM macros: four 256×8 for A, one per row lane, and four for B, one per column. Eight is a bandwidth floor, not a capacity choice — the array consumes four A and four B bytes per compute cycle from 1RW 8-bit parts, so no depth reduction reduces the count — and both banks are exactly full at 1,024 bytes.

The C buffer is where the area was. It holds $N \times N = 16$ accumulators of 24 bits — 48 bytes — which the predecessor stored as three 256×8 macros used as byte planes, bits [7:0], [15:8] and [23:16] sharing one address bus so a 24-bit word moved in one access. That spent 203,314 μm^2 on 48 bytes in 768, and no configuration of this array would ever use the rest: C is $N \times N$ by construction.

This design folds the planes onto the address axis of one 64×8 macro, addressed as

$$addr = word \times 3 + plane, \quad word \in [0, 16),\ plane \in [0, 3), \tag{2}$$

which uses 48 of 64 words and costs 45,861 μm^2 — a 77% reduction and two fewer macros to route around. Sixty-four is the shallowest depth this SRAM family offers, so this is the floor, not a tuning point. On-chip storage is nine macros and 2,112 bytes; the roughly 280 KB of INT8 weights stream through these tiles from outside.

The cost is that 24 bits no longer move in one access. The bank walks the planes itself: a write holds address and data steady and is acknowledged on the last plane, so writeback takes 48 cycles instead of 16 — $+11.8\%$ against the 256 compute cycles it follows — and a read assembles over three plane accesses plus the macro's registered latency, valid on the fourth cycle after the read enable rises. That latency is a contract every caller is written against (Section V). The wrapper carries a behavioral model for simulation and a blackbox branch for synthesis, power pins left to the PDN rather than tied in RTL; since the macro read has one cycle of latency, the top level delays the PE enable and completion flags by one cycle so the array never accumulates invalid data.

V. Chip-Level Integration

The accelerator exposes 53 signal bits, but the shuttle slot gives only 20 general-purpose digital pads beside 60 analog, 8 supply, clock, and reset pads. Rather than a pin-per-signal map, the core wraps the accelerator behind a synchronous 4-wire serial bridge (SCLK, MOSI, CS_N, MISO) that treats SCLK as data: all three inputs are double-flop synchronized and SCLK edge-detected, avoiding a second clock domain. Frames shift MSB-first while CS_N is low — 2-bit command, 10-bit address, plus 8 data bits on writes: WRITE_A and WRITE_B load the operand buffers, START launches a tile, READ_C returns a 24-bit two's-complement result on MISO, and a rising CS_N aborts safely. Three status flags (busy, done, writeback-done) drive pads directly for scope-visible liveness, so 9 of the slot's 91 pads are used. The folded C buffer is host-visible in one place only: a result assembles over three plane accesses, so the host leaves nine core clocks after the last address bit rather than four — at SCLK $\leq$ clk/8, one idle bit before the read data.

The padring is the wafer.space GF180MCU template [11] for a 2935×2935 μm workshop slot. It shorts the pad and core supplies into one rail on every pad, so the chip is single-supply by construction, and the operating point is 3.3 V for the whole die: logic and SRAM are 3.3 V parts and the foundry I/O cells are characterized at 3.3 V, so every cell runs in a foundry-characterized corner and the I/O levels suit a 3.3 V microcontroller host directly. The predecessor completed this flow with a clean chip-level sign-off, LVS over 71,668 instances included; **it has not yet been re-run against the nine-macro macro**, which is the next step and the reason this paper reports macro-level results.

VI. Software to Hardware Flow

The hardened core is the DLA engine alone; everything around it is a host (Fig. 5). In simulation that is Python plus a Verilog testbench preparing quantized weights, inputs, and reference outputs; on the finished chip, a microcontroller streaming the same tile schedule over the serial protocol.

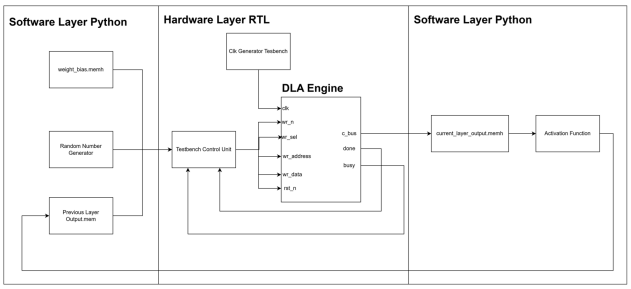

Fig. 5. Hardware and software flow.

A. Quantization and Requantization

Every weight and bias tensor becomes signed 8-bit with one scale factor per tensor, taken from its largest absolute value and recorded in a manifest the testbench reuses. The DLA returns only a tile's integer dot product; the surrounding math is integer on the host, so hardware matches the Python reference exactly. Hidden-layer results are rescaled, biased, ReLU-activated, and requantized to 8 bits; output-layer results go to fixed point, through a tanh lookup table, to a grayscale pixel.

B. Why INT8

TABLE I
NUMERIC FORMATS: ACCURACY AGAINST AN EXACT FP64 RENDER (10 LATENTS, 7,840 PIXELS), AND HARDWARE COST. PE AREA IS SYNTHESIZED ON THE 3.3 V CELLS; ACCUMULATOR WIDTH IS WHAT 256 ACCUMULATED PRODUCTS NEED.

	Pixel error (gray levels)		Weights	PE area	Acc.
Format	mean	max	(KiB)	(μm^2)	(bits)
INT4	22.26	255	138	6,245	15
INT8	**1.22**	**72**	**276**	**15,963**	**23**
INT16	0.01	1	552	47,037	39
FP16	0.06	5	552	—	—

Operand width sets multiplier area, buffer depth, and stream volume at once, so it was chosen by measurement: each format was applied to the same FP32 checkpoint and its render compared against an exact FP64 render of the same latent, against one PE's synthesized area at that width (Table I). INT4 is not viable — a per-tensor scale leaves seven magnitude steps, and 256 accumulated terms of that coarseness destroy the digit. INT16 is essentially exact but costs $2.95\times$ the PE area, a 39-bit accumulator, and double the weight traffic, which doubles time per image since streaming dominates latency (Section IX). FP16 is no more accurate than INT16 for the same memory and a different datapath; none was built, so no area is quoted. INT8 sits at the knee: 1.2 gray levels of mean

error is invisible in a 28×28 digit, the 16-bit product fits the 24-bit accumulator, and the width matches the SRAM word, so buffers need no packing logic.

VII. Verification

The generator maps onto the DLA as tiled dense layers: per tile the four weight rows go to A and the shared input vector to B, the array performs the dot products, and the results land in C, one datapath serving every layer. Seven testbenches pass against Python golden vectors: one GEMM on a single lane; a full $N{=}4/K{=}256$ tile exercising every A and B macro; one layer row; a full 784-pixel image; the pad-level serial protocol driven as external hardware would; and gate-level re-simulations of the last two on the routed nine-macro netlist — inserted buffers and roughly 8,100 antenna diodes included — which reproduce the golden results bit-exactly, closing the gap between "the RTL is correct" and "the taped-out netlist is correct."

Folding the C buffer moved every testbench, and what broke was not obvious. The bank must restart its plane walk when the read *address* changes, not only when the enable drops, or consecutive words interleave and a value assembles from two addresses. The serial bridge needs one wait cycle more than the on-chip caller, because it registers its read enable while the pipeline drives it combinationally, and must re-assert that pulse-type enable in the wait state. Testbenches need five clock edges where RTL callers need four. Each follows from turning one wide access into a sequence of narrow ones, and together they are the real cost of the fold — larger than the 35 flip-flops it added.

The rendered digits (Fig. 6) are limited in quality by the small generator and INT8 quantization, but confirm the full software-to-hardware path runs end to end.

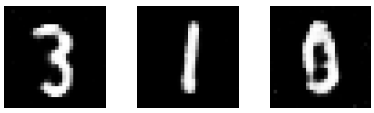

Fig. 6. Digits generated through the hardware path.

VIII. Physical Implementation

The design is implemented with LibreLane [12] on the GF180MCU open PDK [13], hardened as a macro whose GDS/LEF/liberty views a padring integration consumes; everything below is that macro.

A. Single-Supply 3.3 V Library Migration

The buffer SRAM [14] is a 3.3 V IP but the open PDK ships only 5 V cells, so a first version mixed a 5 V core with 3.3 V SRAMs — fine for layout checks, level-shifted on silicon. Migrating to the third-party `gf180mcu_as_sc_mcu7t3v3` native-3.3 V library, which ships nine liberty corners and a LibreLane integration, makes the die single-supply; bring-up surfaced three bugs in its flow integration (a missing timing-corner variable, a synthesis-exclusion filename mismatch, and non-synthesizable `$random` initializers in six sequential-cell models that crashed lint), all fixed at the library source. The 3.3 V cells are far faster than the 5 V ones: the first run at 75 ns closed with $+44$ ns of setup slack, so the clock was retimed to 40 ns (25 MHz).

B. Hardening the Accelerator Macro

TABLE II
Antenna closure on the nine-macro die (40 ns, 1400×1350 μm). All runs are DRC-, LVS-, and timing-clean; only antenna varies. "Structural" marks a violator on a `GEN_B_COLS[1]` SRAM output.

Placement	Density	Nets	Worst	Kind
grid	63	1	$2.18\times$	structural
grid	59	2	$1.73\times$	marginal
grid	56	1	$1.09\times$	marginal
grid	54	1	$1.10\times$	marginal
grid	51	4	$1.49\times$	structural
B/C swap	**56**	**0**	—	**sign-off**
B/C swap	54	0	—	robustness
grid, margin 50	56	0	—	independent

Four configuration points were essential: a synthesis define keeps the SRAM a hard macro (without it the memory is silently built from flip-flops); the macros are placed explicitly on a 3×3 grid with routing channels; the SRAM power pins reach only Metal3, so the macro power connection is made there; and routing runs to Metal5, with cell padding for diode access and diodes inserted at placement, in-router repair disabled.

Antenna closure is where the nine-macro floorplan behaved differently, and that difference is this paper's methodological result. In-router repair aborts in this flow — its verification reroute fails pin access on already-placed clock-net diodes — so closure must come from placement. On the eleven-macro design one density perturbation sufficed: it re-rolls which nets route long, and a single nudge cleared the lone marginal net.

Here it failed (Table II): five densities across a twelve-point span, none reaching zero, settling into a basin at 54–56 with one net near $1.1\times$ and both directions worse. The sweep produced a diagnosis instead. The net names, which a count discards, all pointed one place: `GEN_B_COLS[1]`'s outputs were the $2.18\times$ structural violator at density 63 and returned at $1.49\times$ and $1.18\times$ at 51, while its eight neighbours never violated once. That macro sat in the grid corner furthest from the array, so its column broadcast crossed the whole die — a placement error wearing an antenna costume. Swapping it into the centre and banishing the C buffer (48 bytes, a short net, never a violator) to the corner gave **zero violations** at no cost: $+16.46$ against $+16.11$ ns of setup slack, 145 fewer instances.

Two checks say fix, not lucky re-roll: the same swap is clean at density 54, where the unswapped floorplan sat at $1.10\times$, and holding the floorplan fixed while raising the pre-route diode margin from 25 to 50 reaches zero from the opposite direction — two unrelated levers converging on those broadcast nets. The lesson: *when a density sweep plateaus, stop sweeping and read the net names.* A violator surviving every density is not a lottery; the sweep's real output is its identity, not its count.

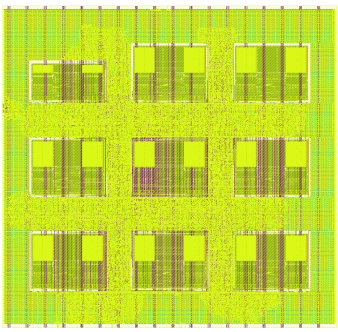

Fig. 7. Hardened accelerator macro, 1375×1325 μm, with the nine SRAM macros on a 3×3 grid. The smaller block at the top left is the folded 64×8 C buffer, banished to the corner so that the antenna-violating B-buffer macro could take the centre slot.

With antenna closed, the die was squeezed. Runs at 1400×1350, 1375×1325 and 1350×1300 μm keep the swapped topology and scale only the die and macro coordinates, so channels tighten and nothing re-rolls. The middle size is promoted: 25 μm off each edge cost 0.22 ns of a 16 ns margin and *saved* 2,171 instances — looser placement carries more buffering, so tightening pays twice. The smallest is past the knee, two nets barely over at $1.04\times$ and $1.01\times$ and a density nudge worsening both directions ($1.75\times$ at 57, $1.31\times$ at 55): congestion, not a re-roll, so the squeeze stopped there.

Fig. 7 shows the resulting layout and Table III the sign-off: zero Magic DRC errors, zero Netgen LVS errors, zero GDS XOR differences, zero violating antenna nets and pins, and the 40 ns clock met at all nine STA corners. The nominal 133 mW is almost entirely internal power estimated with the flow's default switching activity, and should be read as a conservative tool estimate rather than a measurement.

C. Area and Timing Detail

TABLE III
SIGN-OFF AGAINST THE ELEVEN-MACRO PREDECESSOR. BOTH ARE HARDENED ON GF180MCU 180 NM AT 3.3 V WITH THE GF180MCU_AS_SC_MCU7T3V3 CELLS AT 40 NS, AND BOTH CLOSE WITH 0 DRC, 0 LVS, 0 XOR, AND 0 ANTENNA VIOLATIONS. POWER IS A TOOL ESTIMATE.

	11 macros	9 macros	Change
Die (μm)	1600×1500	1375×1325	
Die area (mm^2)	2.400	1.822	-24%
Core area (mm^2)	2.326	1.761	-24%
SRAM area (μm^2)	745,485	588,032	-21%
C buffer (μm^2)	203,314	45,861	-77%
Instances	93,172	79,676	-14%
Power (mW)	162	133	-18%
Wirelength (mm)	867	812	-6%
Writeback (cycles)	16	48	$+200\%$
Compute per image (ms)	3.54	3.95	$+11.7\%$
Setup slack, worst (ns)	$+15.12$	$+16.24$	better
Hold slack, worst (ns)	$+0.150$	$+0.116$	thinner
Clock skew, worst (ns)	0.65	0.44	-32%
Critical path (ns)	24.88	23.76	-4%

The core area shows what the fold achieved and what it did not. Nine macros take 0.588 mm^2 against 0.745 mm^2: a 21% cut of SRAM area from a 77% cut of one buffer. The floorplan stays memory-dominated — macros 33.4% of the core against 21% for all placed cells — since the eight A and B macros are a bandwidth floor and cannot be folded. Arithmetic is a minority of the logic: 0.242 mm^2 combinational against 0.038 mm^2 of well taps, 0.036 mm^2 of 8,145 diodes, and 39.0% fill and decap. Sequential area rises from 406 to 441 flip-flops, the plane-walking state machine — 35 flip-flops bought 157,453 μm^2 of macro.

Timing signs off at nine corners — three PVT points crossed with minimum, nominal, and maximum interconnect — with zero violations, setup set by the slow corner (SS, 125 °C, 3.0 V; $+16.24$ ns) and hold by the fast one (FF, -40 °C, 3.6 V; $+0.116$ ns). The area reduction *improved* setup, $+16.24$ against $+15.12$ ns, because a smaller die shortens the nets the critical path runs on: SRAM read, operand multiplexing, MAC, accumulator register — putting A and B in SRAM is what places a macro read on a combinational path, and at the slow corner its clock-to-output is 12.6–15.0 ns of the total. The remaining slack puts the true critical path at 23.76 ns, closing near 42 MHz; 25 MHz is a deliberate $1.7\times$ guard band on a first tapeout in an unfamiliar library. Hold, the thinnest number in the sign-off, is set by clock-tree skew (0.44 ns), not the period.

Table III collects the comparison: the fold pays on every physical axis at once — 24% of the die, 21% of the SRAM area, 14% of the instances, 18% of the power — and improves setup margin rather than trading against it. It costs cycles in one place only: writeback grows from 16 to 48 cycles per transaction, 11.7% more compute per image and 0.04% of end-to-end latency (Section IX).

IX. PERFORMANCE

A. Latency Breakdown

TABLE IV
END-TO-END LATENCY FOR ONE 28×28 IMAGE AT A 25 MHZ CORE CLOCK AND SCLK OF 3.125 MHZ (CLK/8). ON-CHIP CYCLES ARE MEASURED FROM THE FULL-IMAGE SIMULATION; LINK TIMES FOLLOW FROM THE PROTOCOL'S FRAME LENGTHS AT THAT RATE.

Stage	Frames	Time (ms)	Share
Weight tiles → A buffer	331,776	2,123.4	98.85%
Input vector → B buffer	768	4.9	0.23%
START commands	324	1.2	0.06%
Compute + writeback (on chip)	324 tiles	4.0	0.18%
Result readback from C	1,296	15.3	0.71%
Total		2,149	0.47 img/s

The array performs 16 MAC operations per cycle, a peak of 0.8 GOPS at 25 MHz; quoting only that would be misleading, so this section separates what the array costs from what feeding it costs. On-chip cycle counts are instrumented per controller state in the full-image simulation; link times follow from the frame lengths of Section V.

Per tile the engine is busy 305 cycles: one CLEAR, 256 accumulation, and 48 writeback cycles walking sixteen accumulators through three byte planes. Over 324 tiles that is 98,820 cycles, 3.95 ms at 40 ns — pure compute, 17.9%

of peak, against 3.54 ms and 20.0% for the eleven-macro design; the 11.7% penalty is exactly the writeback growth. The shortfall from peak is structural: the generator evaluates a matrix–*vector* product, so only the four PEs of B's first column produce useful neurons while twelve recompute against unused columns, and batching four latents recovers most of it with no hardware change. Feeding the array costs far more: each tile needs four weight rows across the full $K = 256$ depth, 1,024 bytes, each byte one 20-bit frame at SCLK $\leq$ clk/8 — 3.125 MHz, 6.4 μs per byte (Table IV).

End-to-end latency is therefore **2.149 s per image** (0.47 images/s) against 4.0 ms of arithmetic: the link is 99.8% of it, weight streaming alone 98.8%, an effective 263 kOPS. This settles the area trade — the folded buffer costs 0.41 ms of compute and 0.41 ms of slower reads, together 0.04% of an image, and buys 24% of the die. The bottleneck is not link width but zero weight reuse: a matrix–vector product touches every weight once, so 1.01 bytes cross per MAC however the interface is built, and even an idealized parallel port at one byte per core clock leaves weight loading at 331,776 of 440,903 cycles, 75%. Some traffic is avoidable: the schedule writes the full 256-deep A tile even for L0, whose inputs are 64 wide, so a host pre-zeroing the tail sends 282,624 bytes and finishes in 1.83 s. This is the trade the pad budget forced — 20 pads and 2.1 KB of SRAM against 280 KB of weights — and where future work belongs: a burst write sending the address once removes 12 of every 20 bits.

B. Comparison

TABLE V
COMPARISON WITH REPRESENTATIVE ACCELERATORS. THROUGHPUT IS PEAK (2× MACS × FREQUENCY); EFFICIENCY IS PEAK THROUGHPUT OVER REPORTED POWER. OUR POWER IS A TOOL ESTIMATE.

Design	Tech. (nm)	Area (mm^2)	Peak (GOPS)	Eff. (GOPS/W)	Open flow
DianNao [3]	65	3.02	452	932	No
Eyeriss [4]	65	12.25	67.2	242	No
Wang *et al.* [7]	40	11.97	54.61	567	No
OpenSpike [5]	130	33.3	—	56.8	Yes
This work	180	**1.76**	0.8	6.0	**Yes**

Table V places this work against three fabricated commercial-node accelerators [3], [4], [7] and one open-PDK, open-flow tapeout [5], on peak figures so different scales compare on one basis. The gap reads plainly: two to three orders of magnitude below the commercial parts in throughput, about two in efficiency, almost none of it architectural. A 4×4 array at 25 MHz has $31\times$ fewer multipliers than Eyeriss at $8\times$ the clock, which alone accounts for throughput; the efficiency gap is what 180 nm at 3.3 V costs. The isolating comparison is OpenSpike: at similar scale and clock this design occupies 1.76 mm^2 against 33.3 mm^2, on foundry hard SRAM macros rather than generated memory. What this work offers is reproducibility and density, not performance.

X. CONCLUSION

This work makes a signed-off INT8 GAN-generator accelerator on GF180MCU 24% smaller by questioning its memory rather than its logic. Folding the result buffer's three byte planes onto the address axis of one 64×8 macro cuts that buffer 77%, removes two macros, and allows a 3×3 arrangement on a 1375×1325 μm die drawing 18% less power and closing with *more* setup margin, for 11.7% more compute cycles, 0.04% of end-to-end latency. Both results generalize. Buffer depth and *shape* are separable: byte planes spend one macro per byte of word width, the floorplan pays for macro count rather than occupancy, and folding planes onto the address axis trades width for latency — attractive for any accumulator buffer off the critical path whose depth is fixed by array geometry. And a density sweep is a re-roll, not an optimization: five densities gave no zero but one consistent net name, and acting on it closed antenna in one run. The macro passes DRC, LVS, XOR, and antenna cleanly at 40 ns across nine corners on one 3.3 V supply, verified bit-exactly to a full-image render on the routed netlist. The limits are unchanged: a tile engine with no weight reuse is interface-bound at any width, so batching latents through the unused columns, a burst write, and a wider bus are the levers, none a datapath change. Next is the padring integration, then silicon.

ACKNOWLEDGMENT

The authors thank the open-source EDA community, the GlobalFoundries open PDK program, and the wafer.space chipathon program.

REFERENCES

[1] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. ISCA*, 2017.
[2] C. Silvano *et al.*, "A survey on deep learning hardware accelerators for heterogeneous HPC platforms," *ACM Computing Surveys*, 2025.
[3] T. Chen *et al.*, "DianNao: a small-footprint high-throughput accelerator for ubiquitous machine learning," *Proc. ASPLOS*, 2014.
[4] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: an energy-efficient accelerator for deep convolutional neural networks," *IEEE JSSC*, vol. 52, no. 1, pp. 127–138, 2017.
[5] F. Modaresi, M. Guthaus, and J. K. Eshraghian, "OpenSpike: an OpenRAM SNN accelerator," in *Proc. IEEE ISCAS*, 2023.
[6] B. Jacob *et al.*, "Quantization and training of neural networks for integer-arithmetic-only inference," *Proc. CVPR*, 2018.
[7] C. C. Wang *et al.*, "A 54.61 GOPS 96.35 mW digital logic accelerator for underwater object recognition DNN in 40 nm CMOS," in *Proc. IEEE AICAS*, 2024.
[8] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. NeurIPS*, 2014.
[9] C. Singh, "GAN/VAE pretrained PyTorch models," github.com/csinva/gan-vae-pretrained-pytorch.
[10] "DLA Engine," GitHub, github.com/wlmoi/DLA_Engine.
[11] wafer.space, "GF180MCU project template," github.com/wafer-space/gf180mcu-project-template.
[12] "LibreLane: an open-source RTL-to-GDSII flow," github.com/librelane/librelane.
[13] GlobalFoundries and Google, "GF180MCU open-source PDK," github.com/google/gf180mcu-pdk.
[14] R. T. Edwards, "GF180MCU OCD IP SRAM macros," github.com/RTimothyEdwards/gf180mcu_ocd_ip_sram.